

Projeto prático

Simulador do modelo OSI em uma rede com múltiplos roteadores

Comunicação de Dados

1 Objetivo

Construir um simulador da arquitetura em camadas estudada em sala, capaz de transportar uma mensagem entre dois computadores separados por roteadores e de exibir, passo a passo, o que cada camada de cada dispositivo faz com a unidade de dados que recebe.

A linguagem e a arquitetura do programa são livres. Não há linguagem imposta, estrutura de pastas obrigatória, nomes de classes exigidos nem biblioteca gráfica definida. Uma janela nativa, uma página servida localmente ou uma interface de terminal são todas aceitáveis. O que é avaliado é o comportamento do simulador, e não a forma como ele foi construído por dentro.

O que está em jogo

Um simulador do modelo OSI é um exercício de **separação de escopos**. O par de endereços físicos vale para um enlace e é substituído a cada salto; o par de endereços lógicos vale da origem ao destino e nunca muda; o par de portas vale entre dois processos. Se essa separação estiver correta, cada camada cabe em poucas linhas e o roteador não precisa saber o que é uma porta. Se não estiver, o roteador vira um programa inteiro que conhece tudo, e o simulador deixa de ensinar o que se propõe a ensinar.

2 Escopo

O simulador precisa cumprir os dez requisitos abaixo. Nada além disso é cobrado.

R1 a R5 correspondem ao núcleo da Aula 2: topologia, pilha de camadas, encapsulamento, endereçamento e encaminhamento. R6 completa o tratamento das camadas superiores. R7 e R8 tratam do que aparece na tela e do que é medido, R9 dos casos em que a mensagem não chega, e R10 da forma da entrega. Todos são obrigatórios.

R1. Topologia e endereçamento

A rede simulada é a da Figura 1: três redes locais, quatro roteadores e cinco computadores. Os números sobre os enlaces são custos de encaminhamento.

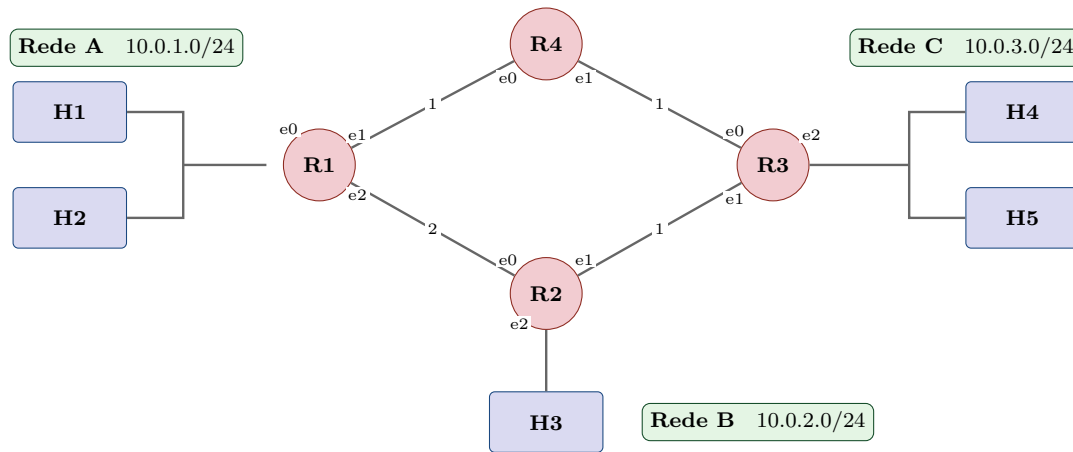


Figura 1: Topologia obrigatória. Os custos indicados sobre os enlaces tornam determinística a escolha de rota.

Os endereços são os da Tabela 1. Os endereços físicos usam um formato reduzido de seis octetos, com o prefixo AA para computadores e BB para roteadores, apenas para facilitar a leitura no registro de eventos.

Dispositivo	Interface	Endereço lógico	Endereço físico
H1	eth0	10.0.1.10	AA:00:00:00:01:0A
H2	eth0	10.0.1.11	AA:00:00:00:01:0B
H3	eth0	10.0.2.10	AA:00:00:00:02:0A
H4	eth0	10.0.3.10	AA:00:00:00:03:0A
H5	eth0	10.0.3.11	AA:00:00:00:03:0B
R1	e0 (Rede A)	10.0.1.1	BB:00:00:00:01:00
	e1 (para R4)	10.0.14.1	BB:00:00:00:01:01
	e2 (para R2)	10.0.12.1	BB:00:00:00:01:02
R2	e0 (para R1)	10.0.12.2	BB:00:00:00:02:00
	e1 (para R3)	10.0.23.2	BB:00:00:00:02:01
	e2 (Rede B)	10.0.2.1	BB:00:00:00:02:02
R3	e0 (para R4)	10.0.34.3	BB:00:00:00:03:00
	e1 (para R2)	10.0.23.3	BB:00:00:00:03:01
	e2 (Rede C)	10.0.3.1	BB:00:00:00:03:02
R4	e0 (para R1)	10.0.14.4	BB:00:00:00:04:00
	e1 (para R3)	10.0.34.4	BB:00:00:00:04:01

Tabela 1: Endereçamento da rede simulada.

A topologia, os endereços e os custos são lidos de um arquivo externo, e não escritos no meio do código. Trocar a rede simulada não pode exigir a edição do código-fonte nem a geração de um novo executável.

R2. A pilha de cada dispositivo

Computadores implementam as sete camadas; roteadores implementam apenas as três primeiras. Cada camada é uma unidade de código separada, com dois métodos de sentido oposto: um que recebe a unidade de dados da camada superior e a entrega à inferior, e outro que faz o caminho inverso. Nenhuma camada acessa diretamente uma camada que não seja adjacente a ela.

N	Camada	Responsabilidade mínima exigida	Unidade
7	Aplicação	Gerar a mensagem e identificar o processo pelo nome.	Mensagem
6	Apresentação	Converter texto em sequência de octetos, registrar o esquema de codificação e cifrar o conteúdo, que só é decifrado na camada 6 do destino.	Mensagem
5	Sessão	Abrir, manter e encerrar o diálogo, atribuindo um identificador de sessão.	Mensagem
4	Transporte	Numerar portas de origem e destino, segmentar a mensagem e remontar os segmentos em ordem no destino.	Segmento
3	Rede	Inserir o par de endereços lógicos e consultar a tabela de encaminhamento.	Pacote
2	Enlace	Inserir o par de endereços físicos do salto, delimitar o quadro e calcular a verificação de erro.	Quadro
1	Física	Converter o quadro em uma sequência de bits e transportá-la pelo enlace.	Bit

Tabela 2: Escopo mínimo de cada camada.

R3. Encapsulamento e desencapsulamento

Na descida, cada camada acrescenta o seu cabeçalho aos dados recebidos e entrega o resultado à camada inferior. Na subida, cada camada remove o seu próprio cabeçalho e entrega o restante à camada superior. Os tamanhos de cabeçalho estão fixados na convenção C1.

A cada salto, o quadro recebido é descartado após a extração do pacote e um quadro **novo** é construído na saída. O pacote, esse sim, atravessa a rede inteira sem alteração. A Figura 2 mostra o percurso completo do caso central.

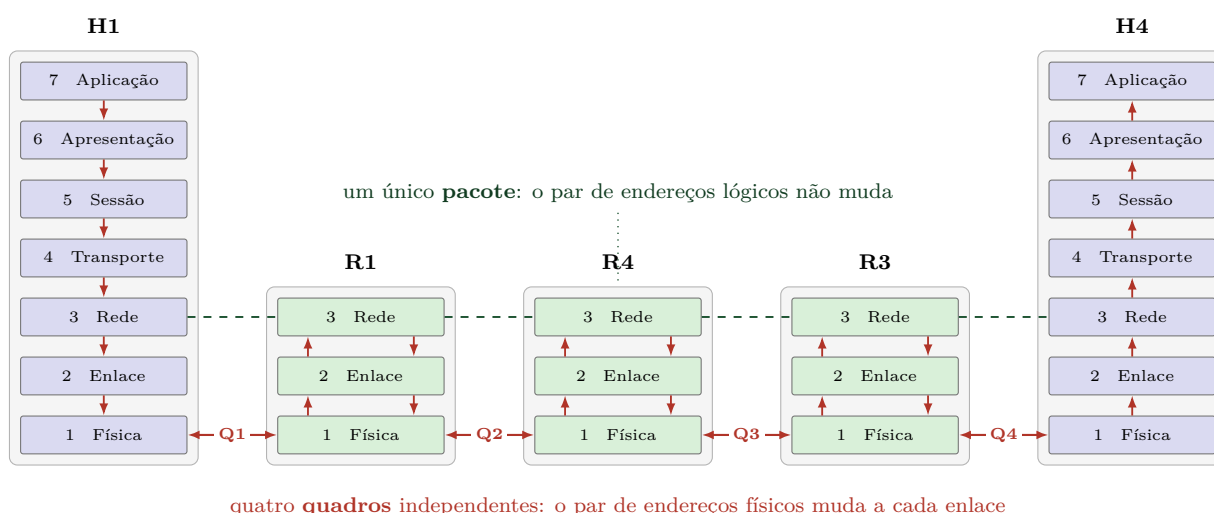


Figura 2: Percurso de H1 até H4 pelo caminho de menor custo. Os roteadores sobem apenas até a camada 3, onde a rota é decidida.

R4. Os quatro níveis de endereçamento

O simulador opera os quatro níveis vistos em aula e mostra, a cada passo, quais mudaram e quais permaneceram fixos.

Nível	Camada	Escopo	Exemplo no caso central
Específico	7	processo	navegador e servidorWeb
Porta	4	processo a processo	5210 e 443
Lógico	3	origem ao destino	10.0.1.10 e 10.0.3.10
Físico	2	salto a salto	muda quatro vezes ao longo do percurso

R5. Encaminhamento

A camada 3 da origem compara o endereço de destino com o próprio prefixo de rede. Se coincidirem, a entrega é direta e há um único quadro. Se não, o pacote é enviado ao roteador da rede local, e cada roteador consulta a própria tabela de encaminhamento para escolher a interface de saída.

As tabelas de encaminhamento decorrem dos custos da Figura 1 e são as da Tabela 3. O simulador pode calculá-las ou lê-las do arquivo de topologia, mas precisa reproduzi-las.

	Rede A (10.0.1.0/24)	Rede B (10.0.2.0/24)	Rede C (10.0.3.0/24)
R1	direta, e0	via R2, e2, custo 2	via R4, e1, custo 2
R2	via R1, e0, custo 2	direta, e2	via R3, e1, custo 1
R3	via R4, e0, custo 2	via R2, e1, custo 1	direta, e2
R4	via R1, e0, custo 1	via R3, e1, custo 2	via R3, e1, custo 1

Tabela 3: Tabelas de encaminhamento de referência.

R6. Segmentação, remontagem e cifragem

A camada 4 divide a mensagem em segmentos numerados quando ela excede o limite da convenção C2, e a camada 4 do destino só entrega à camada 5 depois de remontar todos os segmentos na ordem correta. A camada 6 cifra o conteúdo na origem e o decifra apenas na camada 6 do destino: nenhum roteador e nenhuma camada intermediária tem acesso ao texto claro.

R7. Visualização

A forma de apresentação é livre. Os sete pontos abaixo precisam estar presentes qualquer que seja a escolha de linguagem e de interface.

- V1. **Mapa da rede.** Todos os dispositivos, enlaces e interfaces visíveis ao mesmo tempo, com os nomes da Figura 1 e o caminho percorrido destacado.
- V2. **Pilhas visíveis.** A pilha de cada dispositivo envolvido na comunicação, com a camada ativa em destaque no instante em que processa a unidade de dados. Nos roteadores, a camada 3 destaca-se no momento da decisão de rota.
- V3. **Unidade de dados desenhada.** A unidade corrente como uma sequência de blocos, com os cabeçalhos à esquerda dos dados e o finalizador da camada 2 à direita, acompanhada do nome da unidade.
- V4. **Os dois pares de endereços.** O par lógico e o par físico vigentes na tela ao mesmo tempo, de modo que fique visível que o primeiro permanece constante e o segundo é substituído a cada salto.
- V5. **Controle de execução.** Passo a passo, execução contínua e pausa, com pelo menos três velocidades.
- V6. **Registro na tela.** Área de texto rolável com o registro de eventos de R8.
- V7. **Alternância entre pilhas.** Um controle que troca a exibição entre a pilha OSI de sete camadas e a pilha TCP/IP, agrupando as camadas 5 a 7 em uma única camada de aplicação. A simulação subjacente não se altera.

R8. Registro de eventos e custo do empilhamento

Cada ação de cada camada produz uma linha de registro com cinco campos separados por barra vertical, encerrada pelo tamanho corrente da unidade de dados em octetos. O registro completo pode ser salvo em arquivo.

001	H1	L7	GERA	processo navegador, destino servidorWeb	42 B
002	H1	L6	CODIFICA	octetos UTF-8, conteúdo cifrado	42 B
003	H1	L5	ABRE	sessão S-0001 estabelecida	46 B
004	H1	L4	SEGMENTA	porta 5210 → 443, segmento 1 de 1	54 B
005	H1	L3	ENCAPSULA	10.0.1.10 → 10.0.3.10	74 B
006	H1	L3	ROTEIA	próximo salto 10.0.1.1 pela interface eth0	74 B
007	H1	L2	ENQUADRA	AA:::01:0A → BB:::01:00, quadro Q1	92 B
008	H1	L1	TRANSMITE	736 bits no enlace H1-R1	92 B
009	R1	L1	RECEBE	736 bits do enlace H1-R1	92 B
010	R1	L2	DESENQUADRA	verificação de erro correta, quadro Q1 descartado	74 B
011	R1	L3	ROTEIA	10.0.3.0/24 via R4, custo 2, interface e1	74 B
012	R1	L2	ENQUADRA	BB:::01:01 → BB:::04:00, quadro Q2	92 B

Após a final de cada execução o programa apresenta o tamanho útil da mensagem, o total de octetos transmitidos somando todos os enlaces e a eficiência

$$\eta = \frac{\text{octetos de dados}}{\text{octetos transmitidos}}, \quad \text{sobrecarga} = 1 - \eta.$$

R9. Falhas e descartes

O simulador permite, pela interface, derrubar um enlace, injetar um erro de bit em um enlace e enviar para um endereço fora da topologia. Nos três casos o comportamento esperado está nos cenários E4, E5 e E6 da Seção 4.

R10. Entrega executável

O projeto é entregue como um **programa que abre com dois cliques**. Não será montado ambiente, instalada dependência nem digitado comando para avaliar a entrega.

- o programa precisa funcionar **sem nenhum argumento** de linha de comando, porque um duplo clique não passa argumento algum;
- a escolha do cenário, a troca do arquivo de topologia e o acionamento das falhas de R9 acontecem **dentro** do programa depois de aberto;
- a janela não pode fechar sozinha ao terminar, nem diante de erro: uma mensagem que desaparece antes de ser lida equivale a nenhuma mensagem.

Vale qualquer forma que satisfaça isso: um binário compilado, um executável gerado por um empacotador da linguagem escolhida, ou um arquivo de inicialização associado a um interpretador que já venha instalado. Dependendo do menor número possível de bibliotecas externas: cada uma é uma chance a mais de o programa não abrir na máquina de quem avalia.

Onde fica o arquivo de topologia

Vários empacotadores geram um executável de arquivo único que se descompacta em pasta temporária, e essa pasta deixa de existir quando o programa é encerrado. O programa deve procurar o arquivo de topologia **ao lado do executável** e, só se não o encontrar, recorrer à cópia embutida. Pelo mesmo motivo, o registro de eventos salvo em R8 não pode ser gravado na pasta de descompactação. Se a sua entrega não for empacotada dessa forma, a regra continua valendo: o arquivo de topologia é procurado ao lado do executável.

Teste em uma máquina limpa

Funcionar no computador em que foi escrito não é evidência de nada. Antes de entregar, abra o programa com dois cliques, a partir da pasta descompactada, sem o editor de código aberto e sem ativar nenhum ambiente. É exatamente assim que a entrega será avaliada.

3 Convenções obrigatórias

O comportamento interno é livre, mas os **números** não são. As dez convenções abaixo determinam os valores de referência da seção seguinte. Divergir de qualquer uma delas produz resultados diferentes dos esperados.

- C1. Os cabeçalhos têm tamanho fixo: 4 octetos na camada 5, 8 na camada 4, 20 na camada 3, e 14 de cabeçalho mais 4 de finalizador na camada 2. As camadas 6 e 7 não acrescentam octetos.
- C2. A camada 4 divide em segmentos de no máximo 40 octetos os dados que recebe da camada 5. O último segmento pode ser menor.
- C3. A cifragem da camada 6 não altera o número de octetos da mensagem.
- C4. O roteador não acessa campos das camadas 4 a 7. Se o código de um roteador precisa consultar uma porta ou o nome de um processo, há um erro de modelagem.
- C5. O quadro recebido é descartado após a extração do pacote e um quadro novo é construído na saída. Os quadros são numerados Q1, Q2, e assim por diante, na ordem de transmissão, reiniciando a cada mensagem.
- C6. O par de endereços lógicos é inserido na origem e permanece o mesmo até o destino. O par de endereços físicos vale para um único enlace.
- C7. A decisão de encaminhamento pertence à camada 3. A camada 2 apenas entrega ao vizinho que a camada 3 indicou.
- C8. O custo de um caminho é a soma dos custos dos enlaces que o compõem. Empate é resolvido pelo menor identificador de roteador.
- C9. O quadro com erro é descartado pela camada 2 do receptor, sem notificação às camadas superiores e sem retransmissão.
- C10. Os passos do registro são numerados sequencialmente a partir de 001, um passo por ação de camada, na ordem em que ocorrem.

Onde o cabeçalho de sessão entra na conta

A camada 4 segmenta os dados que recebe da camada 5, e o cabeçalho de sessão já está incluído neles. Uma mensagem de 100 octetos chega à camada 4 com 104 octetos e é dividida em 40, 40 e 24. Tratar o cabeçalho de sessão como parte de cada segmento, em vez de uma vez só, muda todos os totais do cenário E7.

4 Cenários de validação

Os sete cenários abaixo são a verificação objetiva do projeto. Os valores apresentados são os corretos: reproduzi-los, com as convenções da seção anterior, é o que demonstra que o simulador funciona. Todos usam a mensagem de referência de **42 octetos**, exceto E7.

4.1 E1. Entrega direta

H1 envia para H2, ambos na Rede A. A camada 3 reconhece o prefixo comum e entrega sem roteador.

Enlace	Quadro	Endereço físico de origem	Endereço físico de destino	Tamanho
H1-H2	Q1	AA:00:00:00:01:0A	AA:00:00:00:01:0B	92 B

Um quadro, 92 octetos transmitidos, $\eta = 42/92 = 45,7\%$.

4.2 E2. Entrega indireta, caso central

O processo `navegador`, porta 5210, em H1, envia para o processo `servidorWeb`, porta 443, em H4. O caminho de menor custo é R1, R4, R3, com custo total 2.

Enlace	Quadro	Físico de origem	Físico de destino	Tamanho
H1-R1	Q1	AA:00:00:00:01:0A	BB:00:00:00:01:00	92 B
R1-R4	Q2	BB:00:00:00:01:01	BB:00:00:00:04:00	92 B
R4-R3	Q3	BB:00:00:00:04:01	BB:00:00:00:03:00	92 B
R3-H4	Q4	BB:00:00:00:03:02	AA:00:00:00:03:0A	92 B

O par lógico $10.0.1.10 \rightarrow 10.0.3.10$ é o mesmo nos quatro quadros. Quatro quadros, 368 octetos transmitidos, $\eta = 42/368 = 11,4\%$.

4.3 E3. Demultiplexação

H1 e H2 enviam simultaneamente para o `servidorWeb` de H4, com portas de origem 5210 e 6120 e a mesma porta de destino 443. A camada 4 de H4 separa os dois fluxos pela porta de origem e entrega cada um à sua sessão.

Oito quadros, 736 octetos transmitidos, 84 octetos de dados, $\eta = 11,4\%$. Duas sessões distintas, S-0001 e S-0002.

4.4 E4. Falha de enlace

O enlace R1-R4 é derrubado pela interface. O caminho de H1 até H4 passa a ser R1, R2, R3, com custo total 3.

Enlace	Quadro	Físico de origem	Físico de destino	Tamanho
H1-R1	Q1	AA:00:00:00:01:0A	BB:00:00:00:01:00	92 B
R1-R2	Q2	BB:00:00:00:01:02	BB:00:00:00:02:00	92 B
R2-R3	Q3	BB:00:00:00:02:01	BB:00:00:00:03:01	92 B
R3-H4	Q4	BB:00:00:00:03:02	AA:00:00:00:03:0A	92 B

O número de quadros e a eficiência não mudam em relação a E2: o que muda são os endereços físicos e o custo do caminho. É esse contraste que o cenário existe para mostrar.

4.5 E5. Destino inalcançável

H1 envia para 10.0.9.10, endereço que não pertence a nenhuma rede da topologia. O quadro Q1 chega a R1, que não encontra rota e descarta o pacote na camada 3, com registro explícito do descarte.

Um quadro transmitido, 92 octetos, nenhuma entrega, $\eta = 0$.

4.6 E6. Erro de transmissão

Durante E2, um bit do quadro Q3 é alterado no enlace R4-R3. A camada 2 de R3 detecta a inconsistência pela verificação de erro, descarta o quadro e registra o descarte. Nenhuma camada superior de R3 é acionada.

Três quadros transmitidos, 276 octetos, nenhuma entrega, $\eta = 0$. O registro não pode conter nenhuma linha de camada 3 em R3.

4.7 E7. Mensagem longa

H1 envia para H4 uma mensagem de 100 octetos. A camada 5 acrescenta 4 octetos, e a camada 4 recebe 104 octetos e os divide em três segmentos de 40, 40 e 24 octetos.

Segmento	Dados	Quadro resultante	Quadros no percurso
1 de 3	40 B	86 B	4
2 de 3	40 B	86 B	4
3 de 3	24 B	70 B	4

Doze quadros, 242 octetos por enlace, 968 octetos transmitidos ao todo, $\eta = 100/968 = 10,3\%$. A camada 4 de H4 só entrega à camada 5 depois de receber os três segmentos e remontá-los em ordem.

4.8 Quadro resumo

	Cenário	Quadros	Octetos transmitidos	Eficiência
E1	Entrega direta	1	92	45,7%
E2	Entrega indireta	4	368	11,4%
E3	Demultiplexação	8	736	11,4%
E4	Falha de enlace	4	368	11,4%
E5	Destino inalcançável	1	92	0
E6	Erro de transmissão	3	276	0
E7	Mensagem longa	12	968	10,3%

A comparação que interessa

E1 e E2 transportam exatamente a mesma mensagem e diferem apenas no número de enlaces. A eficiência cai de 45,7% para 11,4% sem que um único octeto de dado a mais tenha sido enviado. Essa diferença é o custo do empilhamento, e é o que o projeto existe para tornar visível.

5 Entrega via GitHub

A entrega é feita por um repositório no GitHub, seguindo o fluxo de *fork* e *pull request*. O conteúdo esperado em cada documento está descrito no Guia de Documentação, disponibilizado junto a este enunciado.

Repositório oficial do trabalho:

<https://github.com/viniciussbrg/simulador-osi>

Passo a passo da entrega

1. Fazer o fork do repositório oficial:

- acessar o repositório oficial no endereço acima;
- clicar no botão **Fork**, no canto superior direito;
- na tela seguinte, manter o nome sugerido e clicar em **Create fork**;
- o GitHub cria uma cópia na conta do aluno, em github.com/<aluno>/simulador-osi.

2. Clonar o fork na máquina local. Apenas um integrante do grupo precisa fazer isso:

```
git clone https://github.com/<aluno>/simulador-osi.git
```


3. **Desenvolver o trabalho no fork do grupo**, seguindo a estrutura de pastas descrita adiante e o conteúdo descrito no Guia de Documentação;

4. **Enviar as alterações ao longo do desenvolvimento**, e não em um único envio no final:

```
git add .
git commit -m "Mensagem descritiva da alteração"
git push origin main
```

5. **Abrir um Pull Request** ao concluir o trabalho:

- no fork do aluno, clicar em **Contribute** → **Open pull request**;
- verificar se o destino é a **branch do seu grupo** no repositório oficial, e não a **main**;
- definir o título no formato **Entrega - Grupo X - Nome dos integrantes**;
- adicionar uma breve descrição com os nomes dos integrantes;
- clicar em **Create pull request**.

6. **Aguardar a revisão**. Após aprovação, o trabalho é incorporado ao repositório oficial na branch dedicada do grupo, preservando a autoria de todos os commits.

Organização do repositório oficial

A figura abaixo mostra como o repositório oficial e os forks dos grupos se relacionam ao longo do trabalho.

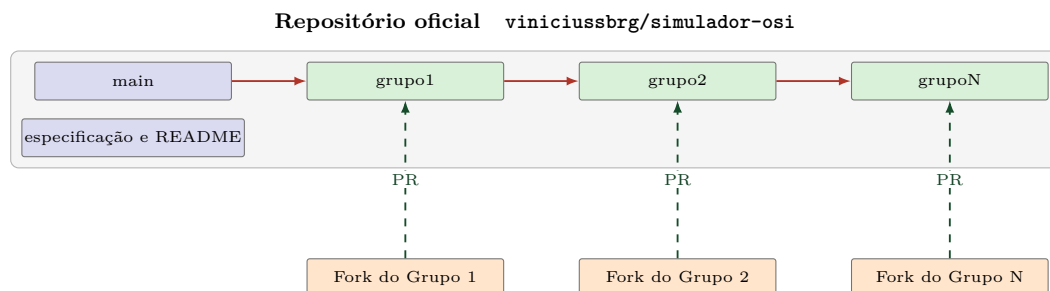


Figura 3: Cada grupo abre o Pull Request do seu fork para a branch do próprio grupo no repositório oficial. A **main** guarda apenas a especificação.

Estrutura sugerida dentro do fork do grupo

O projeto deve ser autocontido: quem entrar na pasta consegue executar e entender o trabalho sem depender de arquivos externos.

```
simulador-osi/
|- README.md                (visão geral, integrantes, índice)
|- SimuladorOSI.exe         (o arquivo do duplo clique)
|- topologia.json           (rede simulada, ao lado do executável)
|- fonte/                  (código-fonte completo)
|- registros/              (registros de eventos dos sete cenários)
'- docs/
   |- tutorial_execucao.pdf  (como abrir e executar)
   |- tutorial_uso.pdf       (como operar cada função)
   |- documentacao_projeto.pdf (descrição técnica)
   '- midia/                (capturas de tela referenciadas nos PDFs)
```

Não altere a branch de outro grupo

Cada grupo trabalha exclusivamente no próprio fork e abre o Pull Request para a própria branch. Fazer commit direto no repositório oficial, ou abrir Pull Request para a **main** ou para a branch de outro grupo, faz a entrega ser devolvida sem análise.

6 Referência

FOROUZAN, B. A. *Comunicação de Dados e Redes de Computadores*. 6. ed. Porto Alegre: AMGH, 2022. Capítulo 2.